

■ Python Fundamentals ■

Part 1 — Output · Input · Variables · Data Types · Operators

cute notes just for you ■

■ 1. Output — print()

The very first thing we learn in Python is how to display something on the screen using the **print()** function. Think of it as your little helper that shows info, results or formatted text on the console. ■■

Syntax & Examples

```
print("hello world")      # prints: hello world
print(3.14)               # prints: 3.14
print(2 + 3)              # prints: 5
print("hello", "world")   # prints: hello world (same line!)
```

■ Python Character Set

Category	Examples
English Letters	'A' to 'Z' and 'a' to 'z'
Digits	0 to 9
Special Symbols	+ - * / % etc.
Whitespace & Escape	blank space, \t (tab), \n (newline)
Unicode & ASCII	All characters as data & literals

■ 2. Variables

A **variable** is like a cute little labeled box ■ that stores data in your program. You can put something in, change it, or use it later!

```
name = "Shradha"         # str
age  = 30                 # int
PI   = 3.14               # float
word = "artificial intelligence"
print("value of PI =", PI)
```

■ Naming Rules (Identifiers)

- Must **start with a letter (a–z, A–Z)** or underscore (`_`)
- Can contain **letters, numbers, or underscores** after the first character
- Cannot use **Python keywords** (if, for, class ...) as names

```
2name = "Bob"      # ■ starts with a number
class = 10         # ■ "class" is a keyword
```

■ Note

Python is dynamically typed — no need to declare the type. It's also case-sensitive: `age` ≠ `Age`. Indentation = 4 spaces per level.

■ 3. Data Types

A **data type** defines the kind of value a variable can hold. Use **`type()`** to check any variable's type!

Type	Meaning	Example
int	Whole numbers	<code>age = 21</code>
float	Decimal numbers	<code>pi = 3.14</code>
str	Text / sequence of chars	<code>name = "Riya"</code>
bool	True / False	<code>isStudent = True</code>
None	No value (null)	<code>x = None</code>

```
x = 10
print(type(x))          # <class "int">

PI = 3.14
print(type(PI))         # <class "float">

name = "shradha"
print(type(name))       # <class "str">

isTeacher = True
print(type(isTeacher))  # <class "bool">

empty_var = None
print(type(empty_var))  # <class "NoneType">
```

■ Type Conversion & Type Casting

Implicit (automatic) — Python does it for you:

```
a = 5
b = 3.0
print(a + b)  # 8.0 → Python auto-converts to float
```

Explicit (type casting) — you do it yourself:

```
x = 10
y = float(x)  # 10.0
z = str(x)    # "10"
```

Common casting functions: **int()**, **float()**, **str()**, **bool()**, **list()**, **tuple()**

■ 4. Operators

An **operator** is a symbol that tells Python to perform a specific computation on one or more values (called **operands**).

```
print(1 + 3)  # "+" is the operator, 1 & 3 are operands
```

■ Arithmetic Operators

Operator	Name	Example (a=5, b=10)	Result
+	Addition	a + b	15
-	Subtraction	a - b	-5
*	Multiplication	a * b	50
/	Division	a / b	0.5
%	Modulo	a % b	5
**	Exponent	a ** b	9765625

■ Relational (Comparison) Operators

Operator	Meaning	Example (a=10,b=20)	Result
==	Equal to	a == b	False
!=	Not equal to	a != b	True
>	Greater than	a > b	False
<	Less than	a < b	True
>=	Greater than or equal	a >= b	False
<=	Less than or equal	a <= b	True

■ Assignment Operators

Operator	Equivalent to	Example	Value of x
=	—	x = 10	10
+=	x = x + n	x += 5	15
-=	x = x - n	x -= 3	12
*=	x = x * n	x *= 2	24
/=	x = x / n	x /= 4	6.0
%=	x = x % n	x %= 4	2.0
=	x = xn	x **= 3	8.0

■ Logical Operators

Used to combine boolean values — **and**, **or**, **not**

val1	val2	AND	OR	NOT val1
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

```
x, y, z = 10, 20, 5
print(x > z and y > x) # True   (AND: both must be true)
print(x > y or y > z)  # True   (OR: at least one true)
print(not (x > y))     # True   (NOT: flips the result)
```

■ 5. Operator Precedence

When multiple operators appear in one expression, Python follows a strict priority order — **highest first**:

Priority	Operator(s)	Description
1 (highest)	()	Parentheses
2	**	Exponent
3	+x -x ~x	Unary operators
4	* / // %	Multiply / Divide / Floor / Mod
5	+ -	Add / Subtract

6	<< >>	Bitwise shifts
7	&	Bitwise AND
8	^	Bitwise XOR
9		Bitwise OR
10	< <= > >= != ==	Comparison
11	not	Logical NOT
12	and	Logical AND
13	or	Logical OR
14 (lowest)	= += -= ...	Assignment

■ 6. Input — input()

Use **input()** to take user input while the program runs. Whatever the user types is always returned as a **string** — so cast it if you need a number! ■

```
name = input("Enter your name: ")
print(name)
print(type(name))      # <class "str">
```

```
# Adding two numbers from the user
a = int(input("Enter 1st number: "))
b = int(input("Enter 2nd number: "))
print(a + b)
```

■ Practice Problem — Average of Two Numbers

```
a = int(input("Enter 1st number: "))    # e.g. 5
b = int(input("Enter 2nd number: "))    # e.g. 10
avg = (a + b) / 2
print("Average =", avg)                 # 7.5
```

■ Quick Cheatsheet

Concept	Syntax / Key Points
print()	print("text") or print(val1, val2)
Variable	name = value — no type declaration needed
type()	type(var) → returns the data type

Casting	int() float() str() bool()
Arithmetic	+ - * / % **
Comparison	== != > < >= <=
Assignment	= += -= *= /= %= **=
Logical	and or not
input()	variable = input("prompt: ")

■ *Keep Learning & Keep Exploring! You've got this, girlie* ■■■